

Agent API Reference

Overview

Your agent is a class that extends `Agent` and implements a single method: `select_move`. Each turn the grader calls this method, passes you a read-only snapshot of the current game state, and expects a move back.

File: Save your agent as `GameAgent.py` with the class named `GameAgent`.

The `select_move` method

```
def select_move(self, game: GameSnapshot, player: PlayerId) -> tuple | None
```

Parameter	Type	Description
<code>game</code>	<code>GameSnapshot</code>	Read-only snapshot of the current board state
<code>player</code>	<code>PlayerId</code>	Your player identifier for this turn

Return value: A `(start, end)` tuple identifying your move, where `start` and `end` are both `(q, r)` hex coordinate tuples. Return `None` only if you have no legal moves — returning `None` when moves exist forfeits the game immediately.

Moves

A **move** is a `(start, end)` tuple of axial hex coordinates:

```
move = ((2, -3), (4, -5))    # move piece from (2, -3) to (4, -5)
start, end = move              # unpack to work with the two cells
```

Both `start` and `end` are `(q, r)` tuples where `q` and `r` are integers.

PlayerId

Your player identifier. Import from `utility`:

```
from utility import PlayerId

PlayerId.PLAYER_1    # first player
PlayerId.PLAYER_2    # second player
# also PLAYER_3, PLAYER_4, PLAYER_5, PLAYER_6 for larger games
```

You receive your `PlayerId` as the `player` argument to `select_move` every turn. Use it to identify your pieces and your goal triangle.

GameSnapshot reference

All game state is exposed through the `GameSnapshot` object passed to `select_move`. It is **read-only** — you cannot modify it. Every collection it returns is immutable (`MappingProxyType`, `frozenset`, or `tuple`).

Properties

`game.pieces` → `MappingProxyType[tuple, PlayerId]`

All pieces currently on the board, keyed by cell coordinate.

```
# Find all your pieces
my_pieces = [pos for pos, owner in game.pieces.items() if owner == player]

# Find all opponent pieces
opp_pieces = [pos for pos, owner in game.pieces.items() if owner != player]

# Check who occupies a specific cell (None if empty)
owner = game.pieces.get((2, -3))
```

`game.current_player` → `PlayerId`

The player whose turn it is. Inside `select_move` this is always equal to `player`.

`game.players` → `tuple[PlayerId, ...]`

All players in turn order.

```
opponents = [p for p in game.players if p != player]
```

game.goal_triangles → MappingProxyType[PlayerId, frozenset]

Each player's goal triangle — the 10 cells they must fill to win.

```
my_goal    = game.goal_triangles[player]          # frozenset of 10 cells
opp_goal   = game.goal_triangles[opponents[0]]    # opponent's goal

# Check if a cell is inside your goal
if end in my_goal:
    print("This move enters my goal")

# Count how many of your pieces are already home
pieces_home = sum(1 for p in my_pieces if p in my_goal)
```

Important: Once a piece enters your own goal triangle it may not leave. Submitting a move that takes a piece back out of your goal forfeits the game. See [Goal Rules](#) below.

game.cells → frozenset[tuple]

Every valid cell on the board (181 cells in a standard 2-player game).

game.triangles → tuple[frozenset, ...]

All six corner triangles of the star board, as a tuple of frozensets. Triangle indices are fixed by board geometry. You rarely need this directly — use `goal_triangles` and `get_starting_positions` instead.

game.move_history → tuple[dict, ...]

All moves made so far, oldest first. Each entry is a dict with these keys:

Key	Type	Description
"player"	str	String form of the PlayerId who moved

Key	Type	Description
"move"	list	[[q1, r1], [q2, r2]] — start and end cells
"timestamp"	str	ISO 8601 timestamp
"piece_count"	dict	Piece counts per player at the time of the move
"is_swap"	bool	True if this was a swap move (only present on swap moves)
"evicted_to"	list	Cell the swapped opponent piece landed on (swap moves only)
"evicted_owner"	str	String form of the evicted piece's owner (swap moves only)

```
# Get the most recent move
if game.move_history:
    last = game.move_history[-1]
    last_start = tuple(last["move"][0])
    last_end = tuple(last["move"][1])

# Find your last move (for oscillation prevention)
for entry in reversed(game.move_history):
    if entry["player"] == str(player):
        prev_start = tuple(entry["move"][0])
        prev_end = tuple(entry["move"][1])
        break
```

Note: coordinates in `move_history` are stored as lists ([q, r]), not tuples. Convert with `tuple(entry["move"][0])` when needed.

game.initial_pieces → **MappingProxyType[tuple, PlayerId]**

The board layout at the very start of the game. Useful for determining which cells form your starting triangle without calling `get_starting_positions`.

`game.game_over` → `bool`

Whether the game has ended. This will always be `False` when `select_move` is called.

`game.winner` → `PlayerId` | `None`

The winning player, or `None` if the game is ongoing or ended without a winner.

`game.loser` → `PlayerId` | `None`

The player who forfeited, or `None` if no forfeit occurred.

`game.num_players` → `int`

The number of players in this game (2, 3, 4, or 6).

Move generation methods

`game.get_all_legal_moves(player_id=None)` → `list[tuple]`

Returns every legal (`start`, `end`) move for `player_id`. Defaults to `current_player` if omitted. **This is the primary method most agents will use.**

Includes swap moves when the swap conditions are met (see [The Swap Rule](#)).

```
moves = game.get_all_legal_moves()          # your moves this turn
moves = game.get_all_legal_moves(player)     # same, explicit

# Filter to only moves that enter your goal
goal_entries = [
    (s, e) for s, e in moves
    if e in game.goal_triangles[player] and s not in game.goal_triangles[player]
]
```

Warning: The legal move list includes goal-exit moves. It is your responsibility to avoid submitting them. See [Goal Rules](#).

`game.get_legal_moves(pos)` → `frozenset[tuple]`

Returns the set of destination cells reachable from `pos` in one move (slide or any-length hop chain). Useful when you want to evaluate a specific piece's options without iterating all

moves.

```
destinations = game.get_legal_moves((2, -3))

# Check if a specific piece can reach a specific cell
if (4, -5) in game.get_legal_moves((2, -3)):
    print("That move is reachable")
```

game.get_moves_with_paths(pos) → dict[tuple, list]

Returns {destination: path} for every cell reachable from pos, where path is a list of cells from pos to destination inclusive (e.g. [pos, intermediate, ..., destination]).

Useful for hop-aware evaluation — you can inspect which cells the piece passes through, which matters for detecting goal-transit violations.

```
paths = game.get_moves_with_paths((2, -3))

for dest, path in paths.items():
    n_hops = len(path) - 1          # number of hops taken
    passes_through = path[1:-1]    # intermediate cells (excluding start and end)
```

game.get_all_legal_moves_for(player_id, pieces) → list[tuple]

Returns legal moves for player_id on an **arbitrary board state** you supply as a plain dict. The standard snapshot properties (.pieces, .goal_triangles, etc.) remain unchanged — only the move calculation uses the dict you pass.

This is the key method for **lookahead**: simulate a move, then ask what moves would be available next.

```
# Simulate one move and find the opponent's responses
future = dict(game.pieces)          # mutable copy
future[end] = future.pop(start)     # apply your hypothetical move

opponent = [p for p in game.players if p != player][0]
responses = game.get_all_legal_moves_for(opponent, future)
```

game.get_moves_with_paths_for(pos, pieces) → dict[tuple, list]

Path-aware version of `get_all_legal_moves_for` — returns `{destination: path}` for a specific piece position on an arbitrary board state.

Needed when doing multi-ply lookahead and you want to inspect intermediate hop cells (for example to detect a goal-transit violation in a simulated future).

```
future = dict(game.pieces)
future[end] = future.pop(start)
paths = game.get_moves_with_paths_for(some_pos, future)
```

`game.get_starting_positions(player_id)` → `frozenset[tuple]`

Returns the 10 cells that `player_id` occupied at the start of the game. Use this to detect whether a piece has left the starting triangle or to compute evacuation priority.

```
start_cells = game.get_starting_positions(player)

# Find pieces still in the starting triangle
stragglers = [p for p in my_pieces if p in start_cells]

# Find pieces that have advanced past the start
advanced = [p for p in my_pieces if p not in start_cells]
```

Goal Rules

These rules are enforced by the game engine. Violating them ends your game immediately by **forfeit**.

No goal-exit

Once one of your pieces enters **your own** goal triangle it may not leave. Any move that takes a piece back out of your own goal is invalid. The legal move list returned by `get_all_legal_moves()` includes these moves — filtering them out is your responsibility.

No goal-transit

A hop chain that **enters your own goal mid-chain and then exits** in the same move is invalid — even if the final destination is outside the goal. If you are constructing long hop chains, inspect the full path using `get_moves_with_paths()` to verify no intermediate cell crosses into and back out of your goal.

Moving through opponent triangles

Pieces may freely move through any triangle that is **not your own goal** — including opponent starting triangles and opponent goal triangles. A piece passing through or resting in an opponent's triangle can always move out again on a future turn.

The Swap Rule

When all of the following are true, a **swap move** is added to your legal move list automatically:

1. Every cell in your goal triangle is occupied (no empty slots remain)
2. At least one of those cells is occupied by an **opponent** piece
3. You have no regular legal move that places a piece into your goal
4. You have at least one piece **directly adjacent** to your goal triangle (touching the boundary — one step away, no hops)

What a swap does: Your adjacent piece teleports into the opponent-occupied goal cell (no matter how deep it is), and the opponent piece is placed on the cell your piece just vacated. If this completes your goal, you win immediately.

Swap moves appear in `get_all_legal_moves()` as standard `(start, end)` tuples where `end` is occupied by an opponent piece inside your goal. You can submit them exactly like any other move — the engine handles the two-piece movement atomically.

Forfeits

Your game ends immediately by forfeit if you:

- Submit a move that takes a piece **out of your own goal triangle**
- Submit a hop chain that **enters your goal mid-chain and exits**
- Submit a move to an **occupied cell** (unless it is a valid swap move)
- Submit a move for a **piece that isn't yours**
- Submit a move to a **cell not on the board**
- Return **None** when legal moves exist

When you forfeit, your pieces remain on the board for the rest of the game and may affect other players' paths.

Board geometry

The board uses **axial hex coordinates**. Each cell is a (q, r) integer tuple. Six directions are available:

```
(-1, 0)  (0, -1)  (1, -1)
  \      |      /
  .....
  /      |      \
(1, 0)   (0, 1)   (-1, 1)
```

Pieces slide to adjacent empty cells or hop over adjacent pieces to land on the empty cell beyond. Hop chains can continue as long as there are valid hops available.

Hex distance between two cells:

```
def hex_dist(a, b):
    dq, dr = a[0] - b[0], a[1] - b[1]
    return (abs(dq) + abs(dr) + abs(dq + dr)) / 2
```